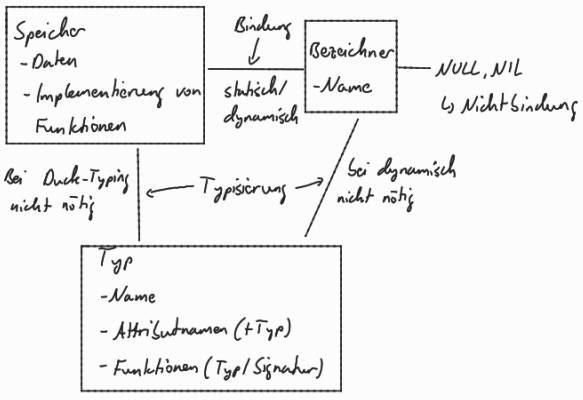
Kevin Küster

Zusammenfassung: Semantik

Semantik

Typisierung

Bezeichner (Identifier) als „Namen" (Variablen, Typen, Klassen, Funktionen, ...)



- Statisch
  - ↳ Wird bestimmt durch Blockstruktur
  - ↳ Zur Compilezeit; nur Programmcode-abhängig
  - ↳ (relativer) Speicher-Pointer, der zur Compile-Zeit berechnet wurde
- Dynamisch
  - ↳ Wird durch Verweiskette rückwirkend bestimmt
  - ↳ (Prüfung) zur Laufzeit
  - ↳ Abhängig von tatsächlichen Objekt-Typen
  - ↳ Speicher-Pointer aus Tabelle (Laufzeitsystem)
- Nicht-Bindungen
  - ↳ NULL, NIL
  - ↳ Vermeidung durch Prüfen (→ NullPointerException)

=> Zuweisung bestimmter Type-Eigenschaften zur Vermeidung von Laufzeitfehlern

- Typisierung: stark ↔ schwach
  - ↳ Wie strong ist Unterscheidung? Umwandlung möglich?
- Typisierung: dynamisch ↔ statisch
  - ↳ Typprüfung zur Übersetzungszeit oder Laufzeit
- Typisierung explizit ↔ implizit
  - ↳ Werden Datentypen explizit genannt oder durch Ableitung ermittelt?

Duck-Typing

Variante der dynamischen Typisierung, die die Anwendbarkeit von Funktionen nicht an Ableitung der Klasse an bestimmte Superklasse knüpft, sondern an Vorhandensein bestimmter Merkmale

Beispiel

Python:
```
for duck in ducks:
    try:
        duck.quak()
    except AttributeError:
        print('Not a duck:', duck)
```

nicht:
```
if isinstance(duck, Duck):
    duck.quak()
```

Nebenläufigkeit

Nebenläufigkeit: Modell/Konzept zur gleichzeitigen Ausführung von Prozessen
Parallelität: Tatsächlich gleichzeitige Ausführung (→ unterschiedliche Hardware)

Basis: Threads

↳ Koordinierung von Threads:

- Shared-state concurrency
  - ↳ Zugriff auf gemeinsame Datenobjekte/Speicherbereiche über Kontrollmechanismen (Monitore)
- Message-passing concurrency
  - ↳ Keine gemeinsamen Datenobjekte, sondern Austausch von Nachrichten

Shared-state Nebenläufigkeit

- Gemeinsame Datenobjekte
- Koordination/Verhinderung nichtdeterministischer Zugriffe
  - Semaphore: Datenstruktur, die aus einer Ganzzahl und atomaren Nutzungsoperationen „Reservieren/Probieren", "Freigeben" besteht
  - Monitore: Programmiersprachliches Konzept zur Synchronisation von Zugriffen zeitlich verschränkt oder parallel laufender Prozesse oder Threads auf gemeinsam genutzten Ressourcen
- Nebenläufigkeit erfolgt explizit durch Programmierer

Data-flow Synchronisation

- Futures werden als Rückgabewerte von Funktionen übergeben
- Funktion wird in neuem Thread gestartet; Aufrufende Funktion läuft weiter
- Sobald Wert der Future gelesen wird, wird anfordernder Thread blockiert, bis das Ergebnis die Future ersetzt
- Alternativ Callback sobald Ergebnis vorliegt
- Wenn eine Nebenläufige Berechnung eine Exception produziert, „fails" die Future und liefert bei Anforderung die Exception

Message-passing Nebenläufigkeit

- Streams, Flows, Channels
- Zwischen Funktionen, die in Threads laufen
  - ↳ automatische Initiierung dieser Threads